

CSL7190: Advanced Computer Architecture and Operating Systems

Minor, IIT Jodhpur, Instructor: Palash Das, Date: 20.02.2026

Time: 120 Minutes

Full Marks: 50

Attempt all questions. Use the answer sheet for rough work and write your final answers neatly and clearly in the space provided on the question paper. Read each question carefully before attempting. **Good Luck!**

1. Fill in the blanks with the appropriate answers based on the levels of transformation.

[5]

- Two processors with different internal designs can run the same program if they implement the same _____ ISA _____.
- Pipelining is a feature of the _____ Microarchitecture _____ layer, not the ISA.
- The logic gates used to build adders and multiplexers belong to the _____ logic _____ layer.
- Adding a new instruction requires modification to the _____ ISA _____ specification first.
- Two CPUs with the same ISA but different cache sizes differ in their _____ Microarchitecture _____.

2. Two programs A and B solve the same computational problem but are written in different programming styles. Program A is compiled with optimization level O2 and runs on processor P1 operating at 1 GHz, while Program B is compiled with optimization level O3 and runs on processor P2 operating at 1.4 GHz. Processor P1 has a 32 KB L1 cache and 256 KB L2 cache, whereas P2 has a 64 KB L1 cache and 512 KB L2 cache. Both processors use a 64-bit architecture and support the same instruction set, although their internal microarchitectures differ. Program A executes 10,000 instructions, of which 20% are branch instructions, 40% are load/store instructions, and the remaining are ALU instructions. The program uses integer arithmetic only and does not involve floating-point operations. Program B executes 12,000 instructions, of which 25% are branch instructions. In Program B, the number of load/store instructions is twice the number of ALU instructions. Assume there are no floating-point instructions in either program. On both processors, branch instructions have an average CPI of 5 and ALU instructions have an average CPI of 1.5. The processors differ only in the CPI of load/store instructions: it is 2 on P1 and 3 on P2. Additionally, due to memory latency effects, 30% of load/store instructions on P2 incur an extra penalty of 2 cycles per instruction. Assume that branch prediction accuracy, cache hierarchy size, compiler optimization level, and word size do not otherwise affect the given CPI values. There is no pipelining overlap; instructions execute sequentially. [10]

(a) Compute the average CPI for both program–processor combinations.

(b) Compute the total execution time of A on P1 and B on P2.

Ans:

(a) Average CPI

Program A on P1

Instruction mix (10,000 instructions):

- Branch = 20% → 2000
- Load/Store = 40% → 4000
- ALU = 40% → 4000

Total cycles =

$$= (2000 \times 5) + (4000 \times 2) + (4000 \times 1.5)$$

$$= 10000 + 8000 + 6000$$

$$= \mathbf{24000 \text{ cycles}}$$

Average CPI = $24000 / 10000 = \mathbf{2.4}$

Program B on P2

Let ALU = x

Load/Store = 2x

Branch = 25% of 12000 = 3000

So:

$x + 2x + 3000 = 12000$

$3x = 9000 \rightarrow x = 3000$

ALU = 3000

Load/Store = 6000

Effective Load/Store CPI on P2:

Base = 3

30% incur +2 cycles → $0.3 \times 2 = 0.6$

Effective CPI = $3 + 0.6 = \mathbf{3.6}$

Total cycles =

$$= (3000 \times 5) + (6000 \times 3.6) + (3000 \times 1.5)$$

$$= 15000 + 21600 + 4500$$

$$= \mathbf{41100 \text{ cycles}}$$

Average CPI = $41100 / 12000 = \mathbf{3.425}$

Clock period of P1 = 1 ns

Clock period of P2 ≈ 0.714 ns

Total execution times of P1 and P2 are 24000ns and 29345.4ns

Name:

Roll:

Department:

3. Consider the arithmetic expression $Z = (A - B) + (C \times D)$, where all operands initially reside in memory and the result must be written back to memory. Assume memory access takes 2 cycles, each ALU operation takes 1 cycle, instruction fetch time is ignored, and there is no pipelining or overlap. For a Stack architecture, operands must be pushed from memory to the stack before operations, and PUSH/POP involving memory costs 2 cycles. For an Accumulator architecture, there is one implicit accumulator register; LOAD/STORE between memory and accumulator costs 2 cycles, and ALU operations involving a memory operand cost additional 2 cycles. For a Register-Memory architecture, ALU instructions may directly use one memory operand (memory access costs 2 cycles, ALU cost is 1 cycle). For a Register-Register (Load-Store) architecture, all ALU operations are performed only on registers, and LOAD/STORE between memory and registers costs 2 cycles. Write the instruction sequence for each architecture, compute the total execution cycles required in each case, and determine which architecture is the fastest. [10]

Stack Architecture

PUSH A
PUSH B
SUB
PUSH C
PUSH D
MUL
ADD
POP Z

Cycles (Memory = 2)

5 memory ops $\times 2 = 10$
3 ALU ops $\times 1 = 3$
Total = 13 cycles

Accumulator Architecture

LOAD A
SUB B
STORE T
LOAD C
MUL D
ADD T
STORE Z

Load + store = 4×2
ALU with mem = 3×3
Total 17 cycles

Register-Memory Architecture

LOAD R1, A
SUB R2, R1, B
LOAD R3, C
MUL R4, R3, D
ADD R5, R2, R4
STORE Z, R5

Load-store = 6
ALU = 9

Total 15 cycles.

Register-Register Architecture

LOAD R1, A
LOAD R2, B
SUB R3, R1, R2
LOAD R4, C
LOAD R5, D
MUL R6, R4, R5
ADD R7, R3, R6
STORE Z, R7

5 LOAD/STORE $\times 2 = 10$
3 ALU $\times 1 = 3$
Total = 13 cycles

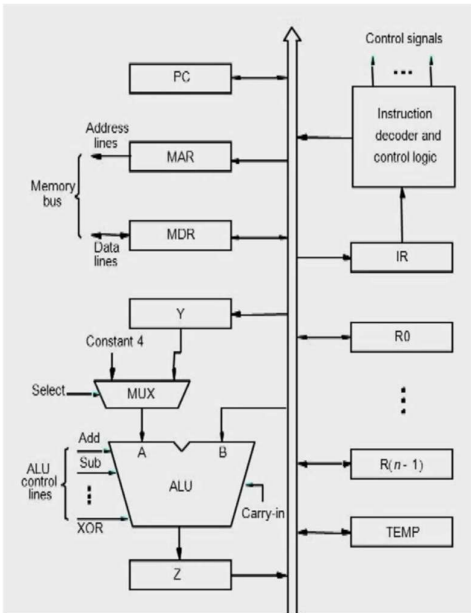
For this configuration, both the stack and Load-store machines are the fastest.

4. Identify the most suitable matches in the table below based on the addressing mode typically generated by a compiler, noting that not all options may apply. [5]

Column A	Column B
1. $x = 5;$	A. Register Indirect
2. $x = A[i];$	B. Scaled Indexed
3. $x = *(p + i);$	C. Auto-increment
4. $x = *p++;$	D. Memory Indirect
5. $x = **(\text{int} **)p;$	E. Immediate

Ans: 1-E, 2-B, 3-B, 4-C, 5-D.

5. Consider a hypothetical non pipelined MIPS processor shown in the datapath diagram (left), with a snapshot of the main memory under a degree of multi-programming of 1 on the right. The OS initializes the PC with 1000, and registers R1 and R2 are preloaded with 20 and 5, respectively. The program begins with a 32-bit R-type instruction, where the 6-bit opcode is followed by 5-bit fields for source register 1, source register 2, and the destination register, starting from the most significant bit. The program encounters abnormal termination immediately after executing the first instruction. The opcode sequence starts with ALU instructions, specifically an addition. The adder requires 3 cycles, the multiplier 8 cycles, and the divider 24 cycles. Trace the execution of this instruction through instruction fetch, decode, register read, execute, and store stages, showing the contents of PC, MAR, MDR, IR, Y, Z, and R3 at each stage, and determine the final result stored in the registers. Assume each stage takes 1 clock cycle (except execution and register read), with register read to Y and the next operand occurring in consecutive cycles, and a clock period of 5 ns. Compute the total CPU time required until the program terminates. [12]



Processor

Address	Content
1000	000000 00001 00010 00011 00000 100000
1004	000000 00100 00001 00101 00000 100010
1008	101011 00101 00100 0000000000001000
1012	101011 00101 00100 0000000000001000
1016	000010 0000000000100000000000000000

Main Memory

Program Address Space

Name:

Roll:

Department:

Ans: IF – 1, ID – 1, RR – 2, EX – 3, STR – 1 = 8 cycles which is equals to 40ns.

PC = 1000 (IF) content won't change as non pipelined

MAR = 1000 (IF) content won't change as non pipelined

MDR = 000000 00001 00010 00011 00000 100000 (IF) content won't change as non pipelined

IR = 000000 00001 00010 00011 00000 100000 (IF) content won't change as non pipelined

Y = 00000000 00000000 00000000 00010100 (20) (RR)

Z = 00000000 00000000 00000000 00011001 (25) (EX)

R3 = 00000000 00000000 00000000 00011001 (25) (EX)

6.

a. Explain the Von Neumann and Harvard architectures, highlighting their respective trade-offs, and describe how they differ from a dataflow machine. [5]

b. Two memory devices are organized differently, as shown in the following two tables. What is the difference in their addressing schemes? [3]

Address	

1000	Byte 0
1001	Byte 1
1002	Byte 2
1003	Byte 3

Address	Data

2000	Word 0 (16 bits)
2002	Word 1 (16 bits)
2004	Word 2 (16 bits)
2006	Word 3 (16 bits)

Ans:

a. The Von Neumann architecture uses a single memory and shared bus for both instructions and data, making it simple and cost-effective but limited by the Von Neumann bottleneck due to shared memory access. The Harvard architecture uses separate memories and buses for instructions and data, allowing simultaneous access and higher performance at the cost of increased hardware complexity. In contrast, a dataflow machine does not follow a sequential program counter-based control flow; instead, instructions execute only when their input data becomes available, enabling high parallelism but requiring more complex hardware and programming models.

b.

Byte-addressable memory: Each memory address refers to a single byte (8 bits). Consecutive addresses point to consecutive bytes. It provides fine-grained access and is commonly used in modern systems.

Word-addressable memory: Each memory address refers to one word (e.g., 16, 32, or 64 bits depending on architecture). Consecutive addresses point to consecutive words. It simplifies hardware but offers less flexible data access compared to byte-addressable memory.